Data Structures

---

4. Function Templates

Function Overloading

- Function to print an array of integer elements

```
void printArray(int* array, int size)
{
    for ( int i = 0; i < size; i++ )
        cout << array[ i ] << “, ”;
}
```

- Function to print an array of characters

```
void printArray(char* array, int size)
{
    for ( int i = 0; i < size; i++ )
        cout << array[ i ] << “, ”;
}
```

Function Overloading

- Function to print an array of doubles

```
void printArray(double* array, int size)
{
    for ( int i = 0; i < size; i++ )
        cout << array[ i ] << “, ”;
}
```

- Now if we want to change the way function prints the array
  - From 1, 2, 3, 4, 5
  - To 1 -2- 3 -4 -5

Function Overloading

- C++ allows the programmer to overload a function name

- Several functions with the same name but different parameter lists
  - Parameter list determines which function will execute.

- Programmer must give the definition of each function

- Can we overload functions with a single code segment?
  - Leave the job of generating code for separate functions for the compiler

C++ Classes

- Consider the Array class that wraps an array of integers

```
class Array {
    int* pArray;
    int size;
  public:
    …
};
```

- What if we want an Array class that wraps arrays of double?

```
class Array {
    double* pArray;
    int size;
  public:
    …
};
```

C++ Classes

- What if we want an Array class that wraps arrays of Boolean?

```
class Array {
    bool* pArray;
    int size;
  public:
    …
};
```

- Now if we want to add a function sum to Array class?
  - We have to change all the three classes

Generic Programming

- Programs containing generic abstractions
- Generic program abstraction (function, class) can be parameterized with a type
  - Abstractions can work with many different types of data
- Algorithms are written in terms of types to-be-specified-later
  - Instantiated when needed for specific types provided as parameters
- Advantages
  - Reusability
  - Writability
  - Maintainability

Templates

- In C++ generic programming is done using templates
- Two types of templates
  - Function Templates
  - Class Templates
- Compiler generates different type-specific copies from a single template

```
template <class Type>
function or class declaration;
```

```
template <typename Type>
function or class declaration;
```

Function Templates

- A function template can be parameterized to operate on different types of data

- Declaration:

```
template< class T >
void funName( T x );
```

```
template< typename T >
void funName( T x );
```

```
template< class T, class U, … >
void funName( T x, U y, … );
```

Example

- function template to print an array having almost any type of data

```
template< typename T >
void printArray( T* array, int size ) {
    for ( int i = 0; i < size; i++ )
        cout << array[ i ] << “, ”;
}
```

```
int main() {
    int iArray[5] = { 1, 2, 3, 4, 5 };
    printArray( iArray, 5 );
    // Instantiated for int[]

    char cArray[3] = { ‘a’, ‘b’, ‘c’ };
    printArray( cArray, 3 );
    // Instantiated for char[]
    return 0;
}
```

Explicit Type Parameterization

- A function template may not have any parameter

```
template <typename T>
T getInput() {
    T x;
    cin >> x;
    return x;
}
```

```
int main() {
    int x;
    x = getInput();             // ??

    double y;
    y = getInput();             // ??
}
```

Explicit Type Parameterization

- A function template may not have any parameter

```
template <typename T>
T getInput() {
    T x;
    cin >> x;
    return x;
}
```

```
int main() {
    int x;
    x = getInput();              // Error!

    double y;
    y = getInput();              // Error!
}
```

Explicit Type Parameterization

- A function template may not have any parameter

```
template <typename T>
T getInput() {
    T x;
    cin >> x;
    return x;
}
```

```
int main() {
   int x;
   x = getInput< int >();

   double y;
   y = getInput< double >();
}
```

User-Defined Specialization

- A template may not handle all the types successfully
- Explicit specializations might be needed for specific type(s)

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

```
int main( ) {
    isEqual( 5, 6 );            // OK
    isEqual( 7.5, 7.5 );        // OK
    isEqual( “abc”, “xyz” );  // Logical Error!
    return 0;
}
```

User-Defined Specialization

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

General Template

```
template< >
bool isEqual< const char* >(const char* x, const char* y ) {
    return ( strcmp( x, y ) == 0 );
}
```

User Specialization

```
int main( ) {
    isEqual( 5, 6 );            // Target: General Template
    isEqual( 7.5, 7.5 );        // Target: General Template
    isEqual( “abc”, “xyz” );  // Target: User Specialization
    return 0;
}
```

Multiple Type Arguments

```
template< typename T, typename U >
T my_cast( U u ) {
    return (T)u;
}
```

```
int main() {
    double d = 10.5674;
    int j = my_cast( d );    //Error
    int i = my_cast< int >( d );
    return 0;
}
```

User-Defined Types

- Besides primitive types, user-defined types can also be passed as type arguments to templates
- Compiler performs static type checking to diagnose type errors

- Example: Consider String class without overloaded operator "=="

```
class String {
    char* pStr;
    . . .
    // Operator "==" not defined
};
```

User-Defined Types

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

```
int main() {
    String s1 = “xyz”, s2 = “xyz”;
    isEqual( s1, s2 );        // Error!
    return 0;
}
```

User-Defined Types

```
class String {
    char* pStr;
    . . .
    friend bool operator ==(const String&, const String& );
};

bool operator ==( const String& x, const String& y ) {
    return strcmp(x.pStr, y.pStr) == 0;
}
```

```
int main() {
    String s1 = “xyz”, s2 = “xyz”;
    isEqual( s1, s2 );        // OK
    return 0;
}
```

Overloading vs. Templates

Function overloading

- Different data types, similar operation
- Example
  - ‘+’ operation is overloaded for different operand types

Function templates

- Different data types, identical operation
- Example
  - A single function template can calculate sum of array of many types

Overloading vs. Templates – Example

```
String operator +( const String& x, const String& y ) {
    String tmp;
    tmp.pStr = new char[strlen(x.pStr) + strlen(y.pStr) + 1 ];
    strcpy( tmp.pStr, x.pStr );
    strcat( tmp.pStr, y.pStr );
    return tmp;
}
```

```
String operator +( const char * str1, const String& y ) {
    String tmp;
    tmp.pStr = new char[ strlen(str1) + strlen(y.pStr) + 1 ];
    strcpy( tmp.pStr, str1 );
    strcat( tmp.pStr, y.pStr );
    return tmp;
}
```

Overloading vs. Templates – Example

```
template< class T >
T sum( T* array, int size ) {
    T sum = 0;

    for (int i = 0; i < size; i++){
        sum = sum + array[i];
    }

    return sum;
}
```

Any Question So Far?